

Python pour le trading — Document 10

Les environnements

1. Qu'est-ce qu'un environnement, et pourquoi s'en soucier

Un projet Python ne tourne presque jamais seul. Il dépend d'une **version de Python** précise et d'un ensemble de **paquets externes** (NumPy, pandas, requests...) eux aussi à des versions précises. Cette combinaison — une version de Python et ses paquets — est ce qu'on appelle un **environnement**. C'est le contexte dans lequel ton code s'exécute.

Un environnement répond à deux besoins. Le premier est l'**isolation** : les paquets d'un projet ne doivent pas perturber ceux d'un autre. Le second est la **reproductibilité** : pouvoir recréer exactement le même contexte ailleurs — sur la machine d'un collègue, sur un serveur de production, ou sur la tienne dans six mois.

Sans environnements, tout s'installe au même endroit partagé : les projets finissent par se gêner, et les résultats deviennent impossibles à reproduire à l'identique. Ce module part de la notion centrale qui prête le plus à confusion — l'**interpréteur** — puis montre le problème concret, les **méthodes** pour créer des environnements, et comment elles se comparent.

***Pourquoi ça compte.** En trading, un même code doit produire le même résultat du poste de développement au serveur de production. L'environnement est précisément l'outil qui garantit cette identité ; le négliger, c'est s'exposer à des écarts inexpliqués entre « ça marche chez moi » et la réalité en production.*

2. L'interpréteur : la notion qui embrouille

Un **interpréteur** est simplement un fichier exécutable python (par exemple `C:\...\projet\venv\Scripts\python.exe`). Un environnement virtuel, ce n'est rien d'autre qu'un dossier contenant un tel `python.exe` accompagné de ses propres paquets. « Utiliser un environnement », c'est faire en sorte qu'un programme appelle CE `python.exe`-là plutôt qu'un autre.

Or « choisir l'interpréteur » se fait à **deux endroits totalement indépendants** — et c'est la source de la confusion la plus fréquente.

1. Dans le terminal. La commande d'activation (`activate`) modifie la fenêtre de terminal pour que, dans CETTE fenêtre uniquement, `python` et `pip` désignent le `python.exe` de l'environnement. L'effet est limité à ce terminal et disparaît quand on le ferme.

2. Dans l'éditeur (PyCharm, VS Code). L'IDE possède son propre réglage de projet, le « Python Interpreter », qui lui indique quel `python.exe` utiliser pour exécuter, déboguer, autocompléter et lister les paquets. Ce réglage est stocké dans la configuration de l'IDE, pas dans le terminal.

Ces deux mécanismes ne communiquent pas. PyCharm ne lit pas l'état d'activation de ton terminal. D'où le paradoxe que tu observes si : ton terminal affiche bien `(.venv)` et `python` y fonctionne, mais PyCharm continue d'afficher « No interpreter selected » — parce que SON réglage à lui n'a jamais été renseigné. Les deux sont des pointeurs distincts vers un `python.exe` ; en régler un ne règle pas l'autre.

La solution est de renseigner l'interpréteur dans PyCharm une bonne fois : **Settings → Project → Python Interpreter → Add Interpreter → Existing**, puis sélectionner le `python.exe` de ton environnement.

```
# l'interpreteur a designer dans l'IDE = le python.exe de l'environnement
```

```
Windows      : <projet>\.venv\Scripts\python.exe
macOS/Linux : <projet>/.venv/bin/python
```

Une fois ce réglage fait, le message disparaît, l'autocomplétion connaît tes paquets, et le terminal intégré de PyCharm active même automatiquement l'environnement à l'ouverture. Le message qui revient malgré une activation signifie donc presque toujours : « l'IDE n'a pas (ou plus) de python.exe valide configuré » — souvent parce que l'environnement a été recréé ou déplacé.

Pourquoi ça compte. Comprendre que le terminal et l'IDE choisissent l'interpréteur séparément évite des heures de « pourtant c'est activé ! ». En pratique, pour chaque projet, on fait deux gestes complémentaires : activer l'environnement dans le terminal, ET pointer l'interpréteur du projet dans l'IDE vers ce même python.exe.

3. Sans isolation : le problème des dépendances

Quand on installe Python, on obtient un environnement **global** : un interpréteur et un seul dossier de paquets (site-packages) partagé par tous les projets. Tant qu'on n'a qu'un projet, cela suffit. Mais dès qu'on en a deux, le conflit surgit : le projet A exige pandas 1.5, le projet B exige pandas 2.2. Un dossier partagé unique ne peut pas contenir les deux versions ; installer l'une casse l'autre. C'est l'**enfer des dépendances** (dependency hell).

```
Environnement GLOBAL (un seul site-packages partagé)
projet_A veut pandas 1.5  --+
projet_B veut pandas 2.2  ---> impossible : une seule version a la fois
```

La parade est universelle : donner à **chaque projet son propre environnement**, isolé, avec ses propres paquets. C'est ce que font, chacun à leur manière, les outils de la section suivante.

Pourquoi ça compte. Une firme exécute plusieurs stratégies, souvent figées sur des versions précises pour ne pas altérer les résultats. L'isolation garantit qu'ajuster une dépendance pour l'une n'aura jamais d'effet de bord sur les autres.

4. Les méthodes pour créer un environnement

Pour pouvoir comparer les outils, on réalise avec **chacun la même tâche**, en quatre étapes identiques : **(1) créer** l'environnement, **(2) l'activer**, **(3) installer pandas**, **(4) figer** le tout pour le reproduire. Seules les commandes changent ; l'objectif est rigoureusement le même.

4.1 venv (+ pip) — l'outil standard

venv est fourni avec Python : rien à installer. Il crée l'environnement à partir de l'interpréteur qui lance la commande (il n'installe donc pas une autre version de Python).

python -m venv .venv	# (1) créer
.venv\Scripts\Activate.ps1	# (2) activer (Windows PowerShell)
source .venv/bin/activate	# (2) activer (macOS / Linux)
pip install pandas	# (3) installer pandas
pip freeze > requirements.txt	# (4) figer les versions

4.2 virtualenv — l'ancêtre de venv

Quasiment identique à venv, mais c'est un paquet externe (à installer) dont venv s'inspire. Un peu plus rapide, compatible avec d'anciennes versions de Python, et capable de cibler un interpréteur déjà installé.

```

pip install virtualenv          # virtualenv n'est PAS fourni avec Python
virtualenv .venv               # (1) créer
.venv\Scripts\Activate.ps1    # (2) activer (identique à venv)
source .venv/bin/activate      # (2) activer (macOS / Linux)
pip install pandas             # (3) installer pandas
pip freeze > requirements.txt  # (4) figer

```

4.3 conda — environnements + paquets, même non-Python

conda gère à la fois l'environnement ET les paquets, y compris des bibliothèques système non-Python (C, Fortran). Il installe lui-même la version de Python demandée et crée des environnements **nommés**, indépendants d'un dossier projet.

```

conda create -n trading python=3.11 pandas  # (1)+(3) crée + installe Python 3.11
et pandas
conda activate trading                      # (2) activer (par son NOM)
conda install pandas                        # (3) (déjà fait ci-dessus, sinon
ainsi)
conda env export > environment.yml          # (4) figer

```

4.4 poetry — tout-en-un, déclaratif

poetry gère l'environnement virtuel **pour toi** et résout les dépendances. On déclare ce qu'on veut dans un `pyproject.toml` ; poetry écrit un `poetry.lock` qui fige tout l'arbre résolu.

```

pip install poetry          # à installer une fois
poetry init                 # (1) génère pyproject.toml (crée l'env au
besoin)
poetry add pandas           # (3) installe pandas ET écrit poetry.lock
poetry run python mon_script.py  # (2) exécuter DANS l'env (sans 'activer' à la
main)
# (4) figer : poetry.lock EST le verrouillage, à versionner avec le code

```

4.5 uv — pip + venv, en bien plus rapide

uv (écrit en Rust) réunit les rôles de pip et venv avec une vitesse remarquable, et peut même télécharger des versions de Python. C'est un remplaçant moderne, de plus en plus répandu.

```

pip install uv              # ou installeur dédié
uv venv                     # (1) créer (très rapide)
.venv\Scripts\Activate.ps1  # (2) activer (ou s'en passer via 'uv run')
source .venv/bin/activate   # (2) activer (macOS / Linux)
uv pip install pandas       # (3) installer pandas
uv pip freeze > requirements.txt  # (4) figer (ou, en mode projet : uv.lock)

```

Pourquoi ça compte. Le même besoin — un environnement isolé avec pandas, reproductible — se règle avec n'importe lequel de ces outils. Savoir les reconnaître et passer de l'un à l'autre, c'est pouvoir rejoindre n'importe quel projet existant sans le casser.

5. Comparaison des méthodes

Les cinq outils visent le même but mais ne se valent pas selon le contexte. Le tableau résume leurs différences ; les paragraphes qui suivent les opposent deux à deux.

Outil	Inclus avec Python	Gère la version de Python	Paquets non-Python	Fichier de repro	Idéal pour
venv	Oui	Non	Non	requirements.txt	Le cas standard ; réflexe par défaut
virtualenv	Non	Partiel*	Non	requirements.txt	Projets hérités, anciennes versions
conda	Non	Oui	Oui	environment.yml	Science des données, libs compilées
poetry	Non	Non	Non	poetry.lock	Applications structurées, repro stricte
uv	Non	Oui	Non	uv.lock	Pip/venv en bien plus rapide ; moderne

* virtualenv peut **cibler** une autre version de Python déjà installée, mais ne l'installe pas lui-même.

venv vs virtualenv : la même chose, à peu de chose près. venv est intégré à Python, virtualenv est externe mais légèrement plus rapide et sait viser un interpréteur déjà présent. Pour un nouveau projet, venv suffit.

venv/virtualenv vs conda : conda va plus loin en installant lui-même la version de Python et des bibliothèques **non-Python** compilées (utile pour des piles scientifiques lourdes). En contrepartie il est plus volumineux et raisonne en environnements nommés. On le choisit surtout en science des données.

venv+pip vs poetry : poetry ajoute la **résolution** des dépendances, un fichier de **verrouillage** et un pyproject.toml déclaratif, et gère l'environnement à ta place. Plus de structure et de garanties, mais davantage à apprendre que le simple venv + pip.

uv vs tous les autres : uv fait ce que font pip et venv mais beaucoup plus vite, sait gérer les versions de Python et produire un verrouillage. C'est le candidat moderne quand la vitesse et la rigueur comptent.

***Pourquoi ça compte.** Le bon outil dépend du projet, pas d'une mode : venv pour l'immense majorité des cas, conda pour le scientifique lourd, poetry/uv pour une application structurée. Savoir pourquoi on choisit l'un plutôt qu'un autre est plus utile que de tous les maîtriser.*

6. Reproduire un environnement : les fichiers

Un environnement n'a de valeur que s'il est reproductible. Chaque méthode a son fichier, mais tous séparent deux idées : **ce que le projet veut** et **ce qui a été réellement installé**.

requirements.txt (venv, virtualenv, uv) : une liste de paquets avec versions épinglées, produite par pip freeze et réinstallée par pip install -r requirements.txt. Simple et universel.

```
pandas==2.2.0
numpy==1.26.4
```

pyproject.toml (poetry, uv) : déclaratif. On y note les dépendances **directes** voulues (par ex. pandas), sans figer tout le détail. Il dit l'intention.

Fichier de verrouillage — poetry.lock, uv.lock : il fige l'arbre **complet** résolu (versions exactes, y compris les dépendances des dépendances). C'est lui qui garantit une reproductibilité parfaite, là où un requirements.txt écrit à la main peut être incomplet. **environment.yml** joue ce rôle côté conda.

La règle : le pyproject.toml décrit **ce que je veux** ; le lock fige **ce qui a été résolu et installé**. On versionne ces fichiers avec le code (jamais le dossier .venv lui-même, qui se recrée).

Pourquoi ça compte. Reproduire un backtest à l'identique exige de figer non seulement pandas, mais toute la chaîne de dépendances. C'est le rôle du fichier de verrouillage : sans lui, une dépendance indirecte mise à jour peut suffire à changer un résultat financier.

7. La démarche recommandée

La gestion des environnements est une habitude à appliquer dès le premier jour d'un projet, pas un correctif de dernière minute.

1. créer un environnement DEDIE au projet (venv par défaut)
2. l'activer dans le terminal (.venv\Scripts\Activate.ps1 ...)
3. pointer l'interpreteur de l'IDE vers ce meme python.exe
4. installer les paquets DANS l'environnement (pip install ...)
5. figer pour reproduire (pip freeze > requirements.txt)
6. ne JAMAIS installer en global pour un projet ; versionner le fichier de repro

Quel outil ? venv + pip pour la grande majorité des cas ; conda si le projet dépend de piles scientifiques ou de bibliothèques non-Python ; poetry ou uv pour une application plus structurée avec verrouillage. Le principe ne change pas : **un environnement isolé par projet, des versions figées, un IDE pointé sur le bon interpréteur.**

Pourquoi ça compte. Adopter ce réflexe dès le départ supprime l'essentiel des problèmes « ça marche chez moi mais pas en production » — exactement l'hygiène attendue là où un code doit tourner de façon identique du poste de développement au serveur de trading.

Récapitulatif

- 1. Environnement** : une version de Python + ses paquets ; il répond à deux besoins, l'isolation et la reproductibilité.
- 2. Interpréteur** : un python.exe précis ; le terminal (activate) et l'IDE (réglage « Python Interpreter ») le choisissent **séparément** — d'où le message de PyCharm malgré un terminal activé.
- 3. Sans isolation** : un site-packages global partagé → conflits de versions entre projets (enfer des dépendances).
- 4. Méthodes** : venv, virtualenv, conda, poetry, uv — la même tâche (créer, activer, installer, figer) avec des commandes différentes.
- 5. Comparaison** : venv (standard), virtualenv (hérité), conda (non-Python), poetry (structuré + lock), uv (rapide + lock).
- 6. Reproduire** : requirements.txt (figé), pyproject.toml (déclaré), lock / environment.yml (arbre complet figé).

Questions de vérification

Essaie de répondre avant de lire la réponse, fournie juste en dessous, en retrait.

Sur l'environnement et l'interpréteur

Q1. Qu'est-ce qu'un environnement, et à quels deux besoins répond-il ?

Réponse. C'est une version de Python associée à un ensemble de paquets à des versions précises. Il répond à l'isolation (les projets ne se gênent pas) et à la reproductibilité (recréer le même contexte ailleurs).

Q2. Pourquoi PyCharm peut-il afficher « aucun interpréteur » alors que le terminal est bien activé ?

Réponse. Parce que le terminal et l'IDE choisissent l'interpréteur séparément. L'activation ne concerne que la fenêtre de terminal ; PyCharm, lui, a son propre réglage « Python Interpreter » qui n'a pas été renseigné. En régler un ne règle pas l'autre.

Q3. Comment faire disparaître ce message dans PyCharm ?

Réponse. Renseigner l'interpréteur du projet : Settings → Project → Python Interpreter → Add → Existing, puis sélectionner le python.exe de l'environnement (.venv\Scripts\python.exe sous Windows, .venv/bin/python sous macOS/Linux).

Sur le problème des dépendances

Q4. Pourquoi l'environnement global pose-t-il problème avec plusieurs projets ?

Réponse. Il n'a qu'un seul site-packages partagé : deux projets exigeant des versions différentes d'un même paquet ne peuvent pas coexister. Installer l'une casse l'autre — l'enfer des dépendances.

Q5. Quelle est la parade universelle ?

Réponse. Donner à chaque projet son propre environnement isolé, avec ses propres paquets, pour qu'une modification dans l'un n'affecte jamais les autres.

Sur les méthodes et leur comparaison

Q6. Quelles quatre étapes sont communes à toutes les méthodes ?

Réponse. Créer l'environnement, l'activer, installer les paquets, puis figer le tout pour le reproduire. Seules les commandes diffèrent d'un outil à l'autre.

Q7. Qu'apporte conda que venv n'offre pas ?

Réponse. conda installe lui-même la version de Python demandée et des bibliothèques non-Python compilées (C, Fortran), et raisonne en environnements nommés. venv se limite à des paquets Python à partir de l'interpréteur courant.

Q8. Qu'ajoute poetry par rapport à venv + pip ?

Réponse. La résolution des dépendances, un pyproject.toml déclaratif et un poetry.lock qui fige tout l'arbre résolu, le tout en gérant l'environnement à ta place — plus de garanties, mais plus à apprendre.

Q9. Différence entre requirements.txt / pyproject.toml et un fichier de verrouillage ?

Réponse. requirements.txt et pyproject.toml expriment ce que le projet veut (dépendances directes) ; le lock (poetry.lock, uv.lock, environment.yml) fige l'arbre complet résolu, garantissant une reproductibilité parfaite.

Ce module complète la série : maîtriser les environnements — et savoir relier le bon interpréteur au terminal comme à l'éditeur — c'est garantir qu'un code qui fonctionne sur le poste de développement fonctionnera aussi en production, à l'identique.